2.º DAW · DESARROLLO DE APLICACIONES WEB

Entorno Cliente

Unidad 2

Programación en el cliente con JavaScript

Variables · tipos · operadores · ámbito · decisiones · bucles

Índice

1 Introducción: ¿qué hacemos en esta unidad? 2

2 Tema 2.1 — Elegir el lenguaje del cliente 2

3 Tema 2.2 — Variables, operadores y tipos de datos 4

4 Tema 2.3 — Ámbitos de las variables 6

5 Tema 2.4 — Conversiones de tipos (tipado débil) 8

6 Tema 2.5 — Tomar decisiones: if, else if, guard clauses 9

7 Tema 2.6 — Bucles: repetir código 10

8 Glosario rápido 12

9 Tips para no perderse en el examen 13

1 Introducción: ¿qué hacemos en esta unidad?

En la Unidad 1 vimos cómo está construida una página web (HTML, CSS, cómo se comunica el navegador con el servidor). En la **Unidad 2** aprendemos a hacer que la página **haga cosas por sí misma** sin pedir permiso al servidor cada vez.

Ese "hacer cosas" lo hace **JavaScript**, el único lenguaje que todos los navegadores entienden de forma nativa (sin instalar nada).

> **IDEA CLAVE**
>
> **Analogía:**
>
> Si HTML es el esqueleto de la página y CSS es la ropa, JavaScript es el cerebro: decide qué ocurre cuando haces clic, valida si escribiste bien el correo en un formulario, muestra u oculta menús, etc.

Los temas de esta unidad son:

Tema	Qué aprendes
2.1	Qué lenguajes existen para el cliente, cómo se escriben identificadores, qué es el transpilador Babel
2.2	Cómo se crean variables, qué operadores usar, cómo compara JavaScript los valores
2.3	Dónde "vive" cada variable (ámbito), qué es la Zona Muerta Temporal, `"use strict"`
2.4	Cómo JavaScript cambia un tipo de dato en otro automáticamente (y cuándo se liará)
2.5	Cómo tomar decisiones en el código (si pasa X, haz Y)
2.6	Cómo repetir código muchas veces (bucles) y cómo salir de ellos

---

2 Tema 2.1 — Elegir el lenguaje del cliente

2.1.1 Qué lenguajes puede entender el navegador

El navegador solo entiende de forma nativa (sin instalar plugins) un lenguaje: **JavaScript** (basado en el estándar ECMAScript).

Categoría	Ejemplos	¿Qué son?
Obsoletos	Flash, Java Applets, VBScript	Requerían instalar plugins y ya no se usan
Estándar nativo	JavaScript	Lo que ejecutan todos los navegadores
Superconjuntos	TypeScript, JSX	Se escriben primero, pero se **transpilan** (traducen) a JavaScript antes de llegar al navegador

> IDEA CLAVE
>
> **Analogía:**
>
> TypeScript y JSX son como escribir en un idioma extranjero y luego pasar el texto por un traductor (Babel) para que el navegador lo entienda.

ECMAScript vs JavaScript

- **ECMAScript** es la especificación oficial (la norma ECMA-262). Define cómo debe comportarse el lenguaje.
- **JavaScript** es la implementación real que hace cada navegador (Chrome usa V8, Firefox usa SpiderMonkey, Safari usa JavaScriptCore).

2.1.2 Identificadores válidos (nombres de variables, funciones, etc.)

Un identificador es el nombre que le pones a una variable, función, constante, etc. Tiene reglas estrictas:

Regla	Ejemplo válido	Ejemplo inválido	Por qué
No empezar por número	`usuario1`	`1usuario`	No puede empezar por dígito
Puede empezar por letra, _ o $	`_total`, `$btn`	—	Estos sí están permitidos al inicio
No puede llevar espacios	`nombreUsuario`	`nombre usuario`	El espacio rompe el nombre
No puede ser palabra reservada	`contador`	`class`, `function`, `if`, `return`	Son palabras del propio lenguaje
Distingue mayúsculas	`total`, `Total`	—	Son dos variables distintas

Ejemplo práctico:

```
let nombreCliente = "Ana";       // válido: camelCase
const MAX_REINTENTOS = 5;        // válido: constante en mayúsculas
let $botonEnviar = document.querySelector("#enviar"); // válido: empieza por $
let _token = "abc123";           // válido: empieza por guion bajo
```

2.1.3 Sensibilidad a mayúsculas (case-sensitive)

JavaScript diferencia mayúsculas de minúsculas en todo: nombres de variables, funciones nativas y palabras reservadas.

```
let totalVentas = 500;
let TotalVentas = 1200;

console.log(totalVentas);   // 500
console.log(TotalVentas);   // 1200
```

Son **dos variables distintas**. Si escribes `Alert` en lugar de `alert`, o `Function` en lugar de `function`, el navegador no lo reconocerá y dará error.

2.1.4 ASI — Inserción automática de punto y coma

JavaScript intenta poner punto y coma al final de las líneas si te olvidas. Esto suena bien, pero puede causar problemas peligrosos:

```
function crearConfiguracion() {
  return
  { modo: "oscuro", puerto: 8080 };
}
```

¿Qué crees que devuelve esta función? Nada (`undefined`), porque JavaScript inserta un `;` tras el `return`:

```
function crearConfiguracion() {
  return;   // <- JavaScript lo interpreta así
  { modo: "oscuro", puerto: 8080 }; // <- esto queda aislado y no se devuelve
}
```

Regla de oro: nunca rompas una línea justo después de `return`. Pon la llave `{` en la misma línea:

```
function crearConfiguracion() {
  return { modo: "oscuro", puerto: 8080 };
}
```

2.1.5 Convenciones profesionales de nombrado

Entidad	Ejemplo	Convención
Variable normal	`nombreCliente`	camelCase (primera minúscula, luego mayúsculas)
Constante global	`MAX_REINTENTOS`	UPPER_SNAKE_CASE (todo mayúsculas y guiones bajos)
Clase	`UsuarioRegistrado`	PascalCase (todo empieza por mayúscula)
Referencia a botón DOM	`$botonEnviar`	prefijo `$`
Propiedad privada	`_tokenSesion`	prefijo `_`

---

3 Tema 2.2 — Variables, operadores y tipos de datos

3.1 Cómo declarar variables

Palabra clave	¿Se puede reasignar?	Ámbito	¿Cuándo usarla?
`var`	Sí	Función	Casi nunca hoy en día (obsoleto)
`let`	Sí	Bloque `{ }`	Para variables que cambian
`const`	No	Bloque `{ }`	Para valores que nunca cambian

Bloque es todo lo que está entre llaves `{ ... }`.

```
let edad = 20;      // puedo cambiarla después: edad = 21;
const IVA = 0.21;   // no puedo hacer: IVA = 0.25; (da error)
```

> ATENCIÓN
>
> **Importante:**
>
> `const` no hace que el contenido sea inmutable. Si es un objeto o array, puedes modificar sus propiedades o elementos, pero no puedes reasignar la variable a otra cosa distinta.

3.2 Operadores lógicos: `||` vs `??`

Operador	Nombre	Cuándo usa el segundo valor
`||`	OR (o)	Si el primero es **falsy** (`0`, `""`, `false`, `null`, `undefined`, `NaN`)
`??`	Coalescencia nula	Solo si el primero es `null` o `undefined`

Ejemplo clave — el descuento del 0 %:

```
let descuento = 0;   // el usuario eligió 0 % de descuento (válido)

let tasaA = descuento || 15;   // da 15, ¡ERROR! El 0 se considera "falso"
let tasaB = descuento ?? 15;   // da 0, CORRECTO: respeta el cero
```

- `||` tira el `0` porque lo considera un valor vacío y pone el 15 por defecto.
- `??` respeta el `0` porque solo salta si el valor es `null` o `undefined`.

3.3 Comparaciones: `==` vs `===`

```
console.log(5 == "5");   // true  (coerción: convierte "5" a número 5)
console.log(5 === "5");  // false (distinto tipo: número vs texto)
```

- `==`: compara valor, pero primero convierte los tipos si son distintos (**cuidado, puede sorprender**).
- `===`: compara valor **y** tipo. No hay conversiones ocultas. **Usa siempre** `===`.

3.4 El operador `typeof`

Te dice qué tipo de dato tiene algo:

```
console.log(typeof 50n);          // "bigint"   (números muy grandes con sufijo n)
console.log(typeof Symbol());     // "symbol"
console.log(typeof null);         // "object"   (error histórico de 1995, se
  conserva)
console.log(typeof NaN);          // "number"   (NaN significa "Not a Number",
  pero sigue siendo tipo number)
console.log(typeof "hola");       // "string"
console.log(typeof true);         // "boolean"
console.log(typeof undefined);    // "undefined"
```

3.5 Operadores especiales: `in`, `delete`, `instanceof`

```js
const alumno = { id: 1001, curso: "2 DAW", activo: true };

console.log("curso" in alumno);        // true   (¿existe la propiedad?)
delete alumno.activo;                  // borra la propiedad activo
console.log(alumno instanceof Object); // true   (¿es alumno un objeto?)
```

3.6 Operadores unarios: `i++` vs `++i`

```js
let i = 2;
console.log(i++);   // imprime 2 (devuelve el valor actual, DESPUÉS suma 1)
console.log(i);     // imprime 3

let j = 2;
console.log(++j);   // imprime 3 (primero suma 1, luego devuelve el valor)
console.log(j);     // imprime 3
```

3.7 BigInt: números muy grandes

Los números normales (`number`) pierden precisión después de `9007199254740991`. Para números más grandes usas el sufijo `n`:

```js
let granNumero = 9007199254740993n;   // BigInt
console.log(typeof granNumero);       // "bigint"
```

---

4 Tema 2.3 — Ámbitos de las variables

El **ámbito** es la zona del código donde una variable existe y se puede leer.

4.1 `var` vs `let` en bucles con `setTimeout`

```js
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log("var:", i), 100);
}
// Imprime: var: 3, var: 3, var: 3

for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log("let:", j), 100);
}
// Imprime: let: 0, let: 1, let: 2
```

- `var`: todas las funciones comparten la **misma** `i`. Cuando se ejecutan los `setTimeout`, el bucle ya terminó y `i` vale 3.
- `let`: cada vuelta del bucle crea una `j` **nueva y propia**. Cada función recuerda el valor de su iteración.

4.2 Zona Muerta Temporal (TDZ)

Con `let` y `const`, no puedes leer la variable antes de su declaración, incluso si existe una variable con el mismo nombre fuera:

```
let usuario = "Carlos";

function configurarPerfil() {
  console.log(usuario);   // ReferenceError!
  let usuario = "Marta";
}
configurarPerfil();
```

El `let` de dentro "cubre" al global desde el principio de la función, pero está en su **Zona Muerta Temporal** hasta la línea donde se declara. Leerlo antes lanza error.

Solución: usa nombres distintos o lee el global con un identificador diferente.

4.3 Sombreado de variables

Puedes declarar una variable con el mismo nombre dentro de una función; solo existe dentro de esa función:

```
const TASA = 0.21;   // global

function calcularTarifa(precio) {
  const TASA = 0.10;   // local, sombrea a la global solo aquí
  return precio * TASA;
}

console.log(TASA);                // 0.21 (la global)
console.log(calcularTarifa(200)); // 20  (usó la local 0.10)
```

4.4 `"use strict"`

Es una directiva que pones al principio del archivo o función para activar reglas más estrictas:

```
"use strict";

saldoSinDeclarar = 500;   // ReferenceError: variable no declarada
```

Sin `"use strict"`, esa misma línea crearía una variable global escondida en `window`, lo cual es peligroso y difícil de depurar.

5 Tema 2.4 — Conversiones de tipos (tipado débil)

JavaScript tiene **tipado débil**: cambia automáticamente un tipo de dato en otro según el contexto. Esto es potente pero peligroso.

5.1 Conversión con + y -

```
console.log(5 + "5");   // "55"   (texto: concatena)
console.log(5 - "5");   // 0      (número: resta)
```

Regla: si hay un texto en una suma +, todo se convierte a texto y se concatena. Con -, se intenta convertir a número y hacer matemáticas.

5.2 Truthy y Falsy

JavaScript trata algunos valores como "falsos" aunque no lo sean:

Falsy (se comporta como falso)	Truthy (se comporta como verdadero)
`false`	Cualquier objeto o array
`0`	Cualquier número distinto de 0
`""` (cadena vacía)	`"0"` (¡cuidado! es texto, no número)
`null`	`[]` (array vacío es truthy)
`undefined`	`{}` (objeto vacío es truthy)
`NaN`	—

Ejemplo peligroso:

```
if ("0") {
  console.log("Esto se imprime");   // SÍ, porque "0" es texto y no está vacío
}
```

5.3 Funciones de conversión

```
Number("123");    // 123
Number("abc");    // NaN
Number(null);     // 0
Number(undefined);// NaN
Number(true);     // 1

String(123);      // "123"
String(null);     // "null"

Boolean(0);       // false
Boolean("0");     // true   (texto con contenido es true)
Boolean([]);      // true   (array vacío es true)
```

5.4 `Number.isNaN()`

`NaN` es el resultado de una operación numérica fallida. Para comprobar si algo es `NaN`, **no uses** `== NaN` (nunca es igual a sí mismo). Usa `Number.isNaN()`:

```
console.log(NaN === NaN);          // false (sorprendente, pero así es)
console.log(Number.isNaN(NaN));    // true
```

5.5 `JSON.parse`

Convierte un texto con formato JSON en un objeto real de JavaScript:

```
let texto = '{"nombre": "Ana", "edad": 25}';
let persona = JSON.parse(texto);
console.log(persona.nombre);   // Ana
```

---

6 Tema 2.5 — Tomar decisiones: `if`, `else if`, guard clauses

6.1 Estructura básica

```
let nota = 7;

if (nota >= 9) {
  console.log("Sobresaliente");
} else if (nota >= 7) {
  console.log("Notable");
} else {
  console.log("Insuficiente");
}
```

6.2 Guard clauses (comprobaciones al principio)

En lugar de anidar muchos `if` dentro de otros, comprueba los errores al principio y sal temprano:

Antipatrón (demasiado anidado):

```
function calcularMatricula(datos) {
    if (datos !== null) {
        if (typeof datos.precioBase === "number") {
            if (datos.precioBase > 0) {
                // ... cálculo
            } else {
                return { valido: false, error: "Precio inválido" };
            }
        } else {
            return { valido: false, error: "Precio no es número" };
        }
    } else {
        return { valido: false, error: "Datos nulos" };
    }
}
```

Con guard clauses:

```
function calcularMatricula(datos) {
    if (!datos || typeof datos !== "object") {
        return { valido: false, error: "Parámetros inválidos" };   // sale aquí
    }
    if (typeof datos.precioBase !== "number" || datos.precioBase <= 0) {
        return { valido: false, error: "Precio debe ser positivo" }; // sale aquí
    }
    // Si llega aquí, ya sabemos que los datos son válidos
    let total = datos.precioBase;
    return { valido: true, total: total };
}
```

Ventajas: menos indentación, más fácil de leer, los errores se tratan al principio y el cálculo final queda limpio al final.

---

7 Tema 2.6 — Bucles: repetir código

Un bucle ejecuta un bloque varias veces mientras se cumpla una condición.

7.1 Tipos de bucles

Bucle	Cuándo usarlo	Iteraciones mínimas
`while`	Cuando no sabes cuántas veces repetir	0
`do...while`	Cuando necesitas ejecutar al menos una vez	1
`for`	Cuando conoces el contador (número de vueltas conocido)	0
`for...of`	Recorrer valores de un array o cadena	0
`for...in`	Recorrer claves de un **objeto** (nunca arrays)	0
`forEach`	Hacer algo por cada elemento de un array, sin cortar	0

7.2 Ejemplos básicos

`while` — **condición al inicio:**

```
let i = 0;
while (i < 3) {
  console.log(i);
  i++;   // actualización obligatoria, o bucle infinito
}
// Imprime: 0, 1, 2
```

`do...while` — **condición al final (siempre al menos una vez):**

```
let codigo;
do {
  codigo = prompt("Introduce un código de 4 dígitos:");
} while (codigo.length !== 4);
```

`for` — **contador conocido:**

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
// Imprime: 1, 2, 3, 4, 5
```

`for...of` — **valores de un array:**

```
const colores = ["rojo", "azul", "verde"];
for (const color of colores) {
  console.log(color);
}
// Imprime: rojo, azul, verde
```

`for...in` — **claves de un objeto (¡NO para arrays!):**

```
const config = { servidor: "localhost", puerto: 8080 };
for (const clave in config) {
  console.log(clave, config[clave]);
}
// Imprime: servidor localhost, puerto 8080
```

CUIDADO

Nunca uses `for...in` con arrays:

los índices llegan como texto (`"0"`, `"1"`) y puede listar basura del prototipo.

7.3 `break` y `continue`

```
for (let i = 1; i <= 10; i++) {
  if (i === 6) break;      // sale del bucle completamente en el 6
  if (i % 2 === 0) continue; // salta los pares, pasa al siguiente
  console.log(i);          // imprime: 1, 3, 5
}
```

7.4 Error frecuente: borrar elementos de un array mientras lo recorres

```
const numeros = [2, 4, 6, 8, 9];
for (let i = 0; i < numeros.length; i++) {
  if (numeros[i] % 2 === 0) {
    numeros.splice(i, 1);   // borra el elemento en la posición i
  }
}
console.log(numeros);   // [4, 8, 9]  ← ¡FALLO! Se salta el 4 y el 8
```

Por qué falla: al borrar con `splice`, los elementos siguientes se desplazan a la izquierda. Cuando haces `i++`, te saltas el elemento que acaba de ocupar la posición borrada.

Solución correcta: recorrer de atrás hacia adelante:

```
const numeros = [2, 4, 6, 8, 9];
for (let i = numeros.length - 1; i >= 0; i--) {
  if (numeros[i] % 2 === 0) {
    numeros.splice(i, 1);
  }
}
console.log(numeros);   // [9]  ← correcto, solo queda el impar
```

---

8 Glosario rápido

Término	Qué significa
Identificador	Nombre de variable, función, constante
Ámbito (scope)	Zona del código donde una variable existe
TDZ	Zona Muerta Temporal: no se puede leer `let`/`const` antes de declarar
Sombreado	Variable local con mismo nombre que una global; oculta a la global
Falsy	Valor que se comporta como falso en un `if`
Coerción	Conversión automática de un tipo a otro
ASI	Inserción automática de punto y coma por JavaScript
Guard clause	Comprobación de error al inicio que devuelve temprano
Complejidad ciclomática	Cuántos caminos diferentes puede seguir una función

9 Tips para no perderse en el examen

1. `==` **vs** `===`**:** siempre usa `===` a menos que se te pida expresamente lo otro.
2. `0` **es falsy:** en un `if`, `0`, `""`, `null`, `undefined`, `NaN` y `false` entran por el `else`. Pero `"0"` (texto) es truthy.
3. `let` **y** `const` **no se redeclaran:** no hagas `let x = 1; let x = 2;` en el mismo ámbito.
4. `splice` **en bucle creciente se salta elementos:** recorre de atrás hacia adelante (`i--`).
5. `return` **en línea separada:** JavaScript inserta `;` tras el `return` y devuelve `undefined`. Nunca rompas la línea tras `return`.
6. `typeof null` **es** `"object"`**:** es un error histórico, no lo lógices, mémorelo.
7. `forEach` **no tiene** `break`**:** si necesitas cortar el bucle, usa `for...of`.
8. `Number.isNaN(NaN)` **es** `true`**;** `NaN === NaN` **es** `false`**.**

> **IDEA CLAVE**
>
> **Fin de la teoría.**
>
> Te recomienda leer este documento dos veces, luego intentar resolver el examen de prueba sin mirar las respuestas, y finalmente comparar.